Common Problems

Parking Lot

By Evan King · Published Dec 15, 2025 · medium

Understanding the Problem

> 🚙 **What is a Parking Lot System?**
> A parking lot system manages vehicle parking across multiple spots. When a vehicle enters, the system assigns an available spot matching the vehicle type and issues a ticket. When the vehicle exits, the system calculates the parking fee based on time spent and frees up the spot for the next customer.

Requirements

When you walk into the interview, you'll probably get something like this:

> "Design a parking lot system where different types of vehicles can park, and the system manages spot assignment and calculates fees upon exit."

Before you start thinking about classes, you need to nail down exactly what you're building. Spend a few minutes asking questions to turn this into something more concrete.

Clarifying Questions

Structure your questions around what the system does, how it handles mistakes, what's in scope, and what might change later.

Here's how the conversation might go:

> **You**: "So when a vehicle enters, the system assigns it a specific spot automatically?"
> **Interviewer**: "Yes, the system assigns an available spot matching the vehicle type and issues a ticket."

You might be thinking, "what kind of parking lot assigns a specific spot when you drive in?" Fair question. Most real parking lots let you pick your own spot. But the assigned-spot version is how this problem gets asked in interviews because it forces you to design the allocation logic. I agree it's goofy, but just roll with it.

> **You:** "What types of vehicles does the system support? Just cars, or do we need motorcycles, trucks, that kind of thing?"
> **Interviewer:** "Three types. Motorcycles, regular cars, and large vehicles like SUVs or vans."

Good. Now you know the vehicle categories and that the system controls spot assignment.

> **You:** "What happens when the vehicle enters? Do they get a ticket, a code, or something else to prove they parked there?"
> **Interviewer:** "They get a ticket with a unique ID. They'll need that ticket to exit."

Now you know the entry flow.

> **You:** "How does pricing work? Is it hourly, flat rate, different rates for different vehicle types?"
> **Interviewer:** "Keep it simple. Hourly rate, same for all vehicles. Round up to the nearest hour and they pay when they leave."

When the interviewer says "keep it simple," that's your signal to not over-engineer. Don't build a complex pricing engine with surge pricing and discounts unless they explicitly ask for it.

> **You:** "What happens if the lot is full when someone tries to enter? Or if they try to exit with an invalid ticket?"
> **Interviewer:** "Reject entry if there's no compatible spot available. For exit, return an error if the ticket is invalid or already used."

Good error handling clarity.

> **You:** "What if someone loses their ticket? Do we need to handle that case?"

> **Interviewer:** "Good question, but let's keep it simple. Just assume they never lose their ticket for now."

> Interviewers notice when you ask about edge cases and failure modes. Questions like "what if they lose the ticket?" or "what happens when the system is full?" show you're thinking about real-world problems. Don't go overboard with 20 edge cases, but getting into the habit of asking "what can go wrong?" signals mature engineering thinking.

> **You:** "Last question. What's out of scope? Are we worrying about payment processing, entrance gates, cameras, that kind of infrastructure?"
> **Interviewer:** "No. Focus on the core logic. Spot assignment, ticket management, fee calculation. Skip the physical hardware, payment systems, and UI."

Perfect. You've scoped out what not to build.

Final Requirements

After that back-and-forth, you'd write this on the whiteboard:

FINAL REQUIREMENTS

```
Requirements:
1. System supports three vehicle types: Motorcycle, Car, Large Vehicle
2. When a vehicle enters, system automatically assigns an available compatible spot
3. System issues a ticket at entry.
4. When a vehicle exits, user provides ticket ID
   - System validates the ticket
   - Calculates fee based on time spent (hourly, rounded up)
   - Frees the spot for next use
5. Pricing is hourly with same rate for all vehicles
6. System rejects entry if no compatible spot is available
7. System rejects exit if ticket is invalid or already used

Out of scope:
- Payment processing
- Physical gate hardware
- Security cameras or monitoring
- UI/display systems
- Reservations or pre-booking
```

Core Entities and Relationships

Now that requirements are clear, you need to figure out what objects make up your system. Look for nouns in your requirements, but don't turn every noun into a class. Some things are just data.

Let's walk through the candidates:

Vehicle - This one seems obvious at first. We're parking vehicles, so we need a Vehicle class, right? But think about it. The vehicle is external to our system. We don't manage it, track it, or care about its state. We only need to know its type (motorcycle, car, large) to match it with a compatible spot. That's a single piece of classification data, not an entity we need to model. Keep it as an enum, not a class.

ParkingSpot - This is a clear entity. A spot has an ID, a type to match vehicle types, and needs to track whether it's occupied. This has both state and behavior, specifically the ability to mark itself as occupied or free.

Ticket - When a vehicle enters, we issue a ticket. That ticket is a record of the parking session. It holds the ticket ID, which spot was assigned, what type of vehicle, and when they entered. Unlike Vehicle (which is external), Ticket is internal state that our system creates and manages. It groups related data about an active parking session. It's worth modeling as a class even though it's just a data holder.

ParkingLot - Something needs to orchestrate the whole system. When a vehicle enters, something needs to find an available spot, generate a ticket, and mark the spot occupied. When a vehicle exits, something needs to validate the ticket, calculate the fee, and free the spot. That's the ParkingLot. It's the entry point and coordinator.

After filtering, we're left with three entities:

Entity	Responsibility
ParkingLot	The orchestrator. Owns all spots, tracks active tickets, assigns spots at entry, validates tickets and calculates fees at exit. This is the only public API for the system.
ParkingSpot	Represents one parking space. Has an ID, a type (motorcycle spot, car spot, large spot), and an occupied flag. Provides methods to check if it's free and to mark it occupied or free. Doesn't know about tickets or pricing, just its own state.

Entity	Responsibility
Ticket	A record of a parking session. Holds ticket ID, which spot was assigned, vehicle type, and entry time. Read-only after creation. No business logic here, just data that ParkingLot needs to calculate fees and validate exits.

The relationships are simple. ParkingLot owns a collection of ParkingSpots. ParkingLot creates Tickets when vehicles enter. ParkingLot tracks active tickets so it can validate them during exit.

Class Design

With our three entities identified, it's time to define their interfaces. What state does each one hold, and what methods does it expose?

We'll work top-down, starting with `ParkingLot` since it's the entry point, then drilling into `ParkingSpot` and `Ticket`.

For each class, we'll ask two questions:

1. What does this class need to remember to enforce the requirements (its state)?
2. What operations does this class need to support (its methods)?

ParkingLot

ParkingLot is the orchestrator. Everything flows through it. Let's derive its state from requirements:

Requirement	What `ParkingLot` must track
"System automatically assigns an available compatible spot"	All parking spots in the lot
"System automatically assigns an available compatible spot"	Which spots are currently occupied
"System issues a ticket at entry"	Active tickets to validate on exit
"Calculates fee based on time spent (hourly)"	The hourly rate for pricing

First, who should track whether a spot is occupied? The spot itself or the parking lot? Let's weigh the options.

Good Solution: Store occupied flag on the spot

Great Solution: Derive occupancy from tickets

Great Solution: Maintain an occupancy index

We're choosing the indexed approach for this problem. It keeps occupancy tracking centralized, gives us O(1) lookups, and sets us up well for the concurrency patterns we'll discuss in the extensibility section. But as we discussed above, the other approaches are equally valid for different priorities.

In the Amazon Locker problem, we track occupancy differently—using an `occupied` boolean flag directly on the Compartment entity instead of maintaining a Set in the orchestrator.

Why the difference? In Amazon Locker, occupancy represents physical presence. A package is physically placed in the compartment, then the token is generated. Even after the token expires, the package is still physically there. That's intrinsic state.

In Parking Lot, occupancy represents assignment. The ticket is issued at the gate (creating the assignment) before the car physically reaches the spot. The spot becomes "occupied" the moment we assign it, not when the car parks. That's relational state.

Both approaches are defensible for both problems. We're making different choices based on how we think about the domain. What matters is understanding the tradeoffs and being able to defend your reasoning, not memorizing which pattern to use when.

When deciding where state belongs, ask: "Is this a property of the entity itself, or a relationship managed by the system?"

- **Intrinsic** (entity owns): ID, size, physical status like BROKEN

> - **Relational** (orchestrator owns): "currently assigned to ticket X", "reserved by user Y"
>
> Occupancy is relational, it's derived from "a ticket references this spot." The orchestrator manages tickets, so it should manage occupancy. Whether you compute it on demand or maintain an index depends on your performance and concurrency needs.
>
> That said, this distinction isn't absolute. In Amazon Locker, we treat occupancy as physical state (a flag on the entity). Here, we're treating it as relational (a Set in the orchestrator). Both approaches work for both problems. The distinction is a helpful mental model, not a hard rule.

This gives us:

PARKINGLOT STATE

```
class ParkingLot:
    - spots: List<ParkingSpot>
    - occupiedSpotIds: Set<String>
    - activeTickets: List<Ticket>
    - hourlyRateCents: long
```

Let's break down why each field is needed and why it belongs in ParkingLot:

Why `spots` belongs here. The lot owns the collection of all parking spots. When a vehicle enters, `ParkingLot` scans this list to find an available compatible spot. This is the central resource that the orchestrator manages. Spots could live elsewhere (maybe a separate SpotManager class), but that just adds indirection. The lot manages spots, so the lot holds spots.

Why `activeTickets` as a list. During exit, our system receives a ticket ID string. We need to look it up to validate it exists and get the entry time for fee calculation. We'll start with a simple list here and refine this in the implementation section.

Why `hourlyRateCents` belongs here. Pricing is a system-level policy that the lot enforces. Different parking lots might charge different rates. Storing it here lets us instantiate multiple ParkingLot objects with different rates if needed. The alternative is hardcoding it globally,

which makes testing harder. We could also pass the rate into `exit()` each time, but then we're forcing callers to know about pricing, which breaks encapsulation.

> Avoid using floating-point types for money. Floats can't represent decimal fractions exactly - they use binary fractions internally, so values like 0.1 can't be stored precisely. This leads to tiny errors that accumulate in calculations. Store the smallest unit (cents, pennies) as an integer instead. $5.47 becomes 547 cents. All calculations stay exact, and you only convert to dollars for display. See 0.30000000000000004.com for examples of this issue across languages.

Now for operations. What actions does the outside world need to perform?

Need from requirements	Method on `ParkingLot`
"When a vehicle enters, system assigns spot and issues ticket"	`enter(vehicleType)` returns a Ticket
"When vehicle exits, validates ticket and calculates fee"	`exit(ticketId)` returns fee amount

That's it. Two methods. The entire public API.

PARKINGLOT

```
class ParkingLot:
    - spots: List<ParkingSpot>
    - occupiedSpotIds: Set<String>
    - activeTickets: List<Ticket>  // We'll refine this to a Map during implementation
    - hourlyRateCents: long

    + ParkingLot(spots, hourlyRateCents)
    + enter(vehicleType) -> Ticket
    + exit(ticketId) -> long
```

The constructor takes a list of spots (configured externally) and a rate. `enter` takes a vehicle type and returns the ticket if successful, throws an error if no spots available. `exit` takes a ticket ID and returns the fee in cents, throws an error if the ticket is invalid.

> Some candidates add a `getAvailableSpots()` or `getParkingStatus()` method thinking they need to expose internal state. Unless the requirements explicitly say you need to query the lot's status, there is no need to add these methods. They violate encapsulation and aren't needed for the core workflow. If the interviewer asks later about monitoring or dashboards, you can add them then.

ParkingSpot

ParkingSpot represents one physical parking space. From requirements:

Requirement	What `ParkingSpot` must track
"System assigns compatible spot"	Spot type (motorcycle, car, large) to match with vehicle type
"When a vehicle exits, user provides ticket ID"	Unique ID for the spot

State:

PARKINGSPOT STATE

```
class ParkingSpot:
    - id: String
    - spotType: SpotType
```

For operations:

Need from requirements	Method on `ParkingSpot`
"System automatically assigns an available compatible spot"	`getSpotType()` returns type
"System issues a ticket at entry"	`getId()` returns spot ID

PARKINGSPOT

```
class ParkingSpot:
    - id: String
    - spotType: SpotType

    + ParkingSpot(id, spotType)
    + getSpotType() -> SpotType
    + getId() -> String
```

ParkingSpot is deliberately simple. It's a pure data holder representing the physical properties of a parking space. It doesn't know about vehicles, tickets, pricing, or even whether it's occupied—that's all managed by ParkingLot.

The enums:

SPOTTYPE

```
enum SpotType:
    MOTORCYCLE
    CAR
    LARGE
```

VEHICLETYPE

```
enum VehicleType:
    MOTORCYCLE
    CAR
    LARGE
```

> We have two separate enums (SpotType and VehicleType) even though they have the same values. This keeps them semantically distinct. A spot type is not the same concept as a vehicle type, even if they happen to use the same labels. If requirements later say "motorcycles can use car spots if motorcycle spots are full," having separate enums makes that logic clearer.

Ticket

Ticket is a record of a parking session. From requirements:

Requirement	What `Ticket` must track
"When a vehicle exits, user provides ticket ID"	Ticket ID string
"Frees the spot for next use"	Which spot the vehicle is in
"System supports three vehicle types"	Type of vehicle (not used in base pricing, but stored for per-type pricing extension)
"Calculates fee based on time spent"	When they entered (needed for fee calculation)

State:

TICKET STATE

```
class Ticket:
    - id: String
    - spotId: String
    - vehicleType: VehicleType
    - entryTimeMs: long
```

Why spotId as a string, not a reference to ParkingSpot? Tickets are records, not navigational objects. They shouldn't reach into the domain model. Storing just the ID keeps them simple and prevents tickets from accidentally calling methods on spots. This is the Law of Demeter in action.

Why entryTimeMs as a long (timestamp in milliseconds)? We need to calculate time spent, which means we need to do arithmetic with time values. We're using a simple long here to keep the pseudocode language-agnostic. In real code, you could also use your language's native time type (Java's `Instant`, Python's `datetime`, JavaScript's `Date`, etc.) which handle timezone and duration calculations properly.

Now we need to decide where fee calculation logic belongs. This is a common design decision that trips up many candidates.

Bad Solution: calculateFee() method on Ticket

Good Solution: computeFee() as a method in ParkingLot

Great Solution: Separate PricingStrategy interface

Given we are designing for a single parking lot, we will go with the Good approach. Ticket has no behavior beyond getters. It's a pure data holder:

TICKET

```
class Ticket:
    - id: String
    - spotId: String
    - vehicleType: VehicleType
    - entryTime: long

    + Ticket(id, spotId, vehicleType, entryTime)
    + getId() -> String
    + getSpotId() -> String
    + getVehicleType() -> VehicleType
    + getEntryTime() -> long
```

All fields are read-only after construction. Once a ticket is issued, it never changes. This makes tickets immutable value objects, which is exactly what you want for records.

Final Class Design

Here's how the system fits together: ParkingLot acts as the orchestrator, receiving entry and exit requests, tracking which spots are occupied, finding available spots, generating tickets, and calculating fees. ParkingSpot is a pure data holder representing physical parking spaces (intrinsic properties only). Ticket is an immutable record created at entry that captures the parking session details needed for fee calculation at exit.

FINAL CLASS DESIGN

```
class ParkingLot:
    - spots: List<ParkingSpot>
    - occupiedSpotIds: Set<String>
    - activeTickets: Map<string, Ticket>
    - hourlyRateCents: long

    + ParkingLot(spots, hourlyRateCents)
    + enter(vehicleType) -> Ticket
    + exit(ticketId) -> long

class ParkingSpot:
    - id: String
    - spotType: SpotType

    + ParkingSpot(id, spotType)
    + getSpotType() -> SpotType
    + getId() -> String

class Ticket:
    - id: String
    - spotId: String
    - vehicleType: VehicleType
    - entryTime: long

    + Ticket(id, spotId, vehicleType, entryTime)
    + getId() -> String
    + getSpotId() -> String
    + getVehicleType() -> VehicleType
    + getEntryTime() -> long

enum SpotType:
    MOTORCYCLE
    CAR
    LARGE

enum VehicleType:
    MOTORCYCLE
    CAR
    LARGE
```

The design maintains a clear **separation of concerns**: orchestration, relational state, and business rules in ParkingLot; intrinsic properties in ParkingSpot; and session data in Ticket.

Implementation

With the class design locked in, we need to implement the core methods. Before starting, check with your interviewer. Some want working code in a specific language, others want pseudocode, some just want you to talk through it. We'll use pseudocode here, but the full implementations in multiple languages are at the end.

For each method, we'll follow a pattern:

1. **Define the core logic** - The happy path that fulfills the requirement
2. **Handle edge cases** - Invalid inputs, boundary conditions, unexpected states

The most interesting methods are `enter` and `exit` on ParkingLot. Those are where the orchestration happens.

ParkingLot

Before we implement the methods, let's refine one design decision. In our class design, we kept `activeTickets` as a list. But during exit, we need to look up a ticket by its ID. While we could scan through the list, using a `Map<String, Ticket>` makes the "lookup by ID" intent explicit and cleaner. The performance difference is negligible at parking lot scale, but the map makes the code more readable.

> I ran a quick benchmark to prove that it doesn't matter from a time complexity perspective. With 200 tickets, map lookup averaged 0.12 microseconds and list scan averaged 1.93 microseconds. The map is technically 16x faster, but we're talking about a 1.8 microsecond difference. That's 0.0000018 seconds. When someone exits the parking lot, the time to physically drive through the gate, or the network latency to process their payment, will dwarf this difference by orders of magnitude.
>
> Both are correct if you can explain your reasoning. The map is slightly cleaner because it makes the "lookup by ID" intent explicit, but the performance argument is irrelevant at this scale and it can be a fun thing to callout in an interview.

REFINED PARKINGLOT STATE

```
class ParkingLot:
    - spots: List<ParkingSpot>
    - occupiedSpotIds: Set<String>
    - activeTickets: Map<String, Ticket>  // Changed from List to Map
    - hourlyRateCents: long
```

Let's start with `enter`. This is where vehicles arrive and get assigned a spot.

Core logic:

1. Find an available spot that matches the vehicle type
2. If no spot found, throw an error
3. Add the spot ID to occupiedSpotIds
4. Generate a unique ticket with spot ID, vehicle type, and current timestamp
5. Store the ticket in activeTickets map
6. Return the ticket

Edge cases:

- No available spots for this vehicle type
- Invalid vehicle type (though the enum prevents this)

Here's the pseudocode. We'll assume standard utility methods `generateUniqueId()` (returns a UUID string) and `getCurrentTimestamp()` (returns Unix time in milliseconds):

ENTER

```
enter(vehicleType)
    spot = findAvailableSpot(vehicleType)
    if spot == null
        return error

    occupiedSpotIds.add(spot.id)

    ticket = createTicket(
        generateId(),
        spot.id,
        vehicleType,
        currentTime()
    )
```

```
    activeTickets[ticket.id] = ticket
    return ticket
```

The flow is straightforward. Find a spot, mark it occupied in our Set, create the ticket, store it, return it. If no spot exists, we throw an error before changing any state.

Now `exit`. This is where vehicles leave and pay.

Core logic:

1. Look up the ticket by ID in activeTickets map
2. If not found, throw an error (invalid or already used)
3. Calculate fee based on entry time and current time
4. Remove the spot ID from occupiedSpotIds (frees the spot)
5. Remove the ticket from activeTickets (prevents double exit)
6. Return the fee

Edge cases:

- Ticket ID doesn't exist (invalid or already used)
- Ticket ID is null or empty
- Time calculation edge cases (what if they stayed 0 minutes? Still charge for 1 hour per "round up" rule)

EXIT

```
exit(ticketId)
    if ticketId == null
        return error

    ticket = activeTickets[ticketId]
    if ticket == null
        return error

    exitTime = currentTime()
    fee = computeFee(ticket.entryTime, exitTime)

    occupiedSpotIds.remove(ticket.spotId)
    activeTickets.remove(ticketId)
```

```
    return fee
```

We validate the ticket exists, calculate the fee, free the spot (by removing from occupiedSpotIds), and remove the ticket. The ticket removal is important. It prevents someone from exiting twice with the same ticket. After exit, the ticket ID becomes invalid.

> We're not distinguishing between "ticket never existed" and "ticket already used." Both return the same error. If you wanted to provide more specific feedback, you could track used tickets in a separate set. But for interview scope, treating both as "invalid ticket" is simpler and good enough.

Let's look at some of the key helper methods, starting with `findAvailableSpot` which is called in the first line of the `enter` method.

FIRST-MATCH LINEAR SCAN

```
findAvailableSpot(vehicleType)
    requiredSpotType = mapVehicleTypeToSpotType(vehicleType)

    for spot in spots
        if spot.spotType == requiredSpotType and spot.id not in occupiedSpotIds
            return spot

    return null

mapVehicleTypeToSpotType(vehicleType)
    if vehicleType == MOTORCYCLE
        return MOTORCYCLE
    if vehicleType == CAR
        return CAR
    if vehicleType == LARGE
        return LARGE
    return error
```

No need to overcomplicate this. We iterate over all spots, check if they match the required spot type and aren't in our `occupiedSpotIds` set. The Set gives us O(1) lookup for each spot. If we don't find any available spots, we return `null` .

> Some candidates try to be clever and add complex allocation logic like "prefer spots near the entrance." Unless the requirements mention this, don't add it. You're burning time on features nobody asked for. If the interviewer wants smarter allocation, they'll ask as an extension question or you can ask them whether they want you to implement it.

What about computeFee?

COMPUTEFEE

```
computeFee(entryTime, exitTime)
    durationMillis = exitTime - entryTime
    durationHours = durationMillis / (1000 * 60 * 60)

    // Round up to nearest hour (5 minutes becomes 1 hour)
    if durationMillis % (1000 * 60 * 60) > 0
        durationHours++

    return durationHours * hourlyRateCents
```

We calculate time spent, convert to hours, round up (any partial hour counts as a full hour), and multiply by the rate. Because we round up, someone who parks for 5 minutes gets charged for 1 hour — no separate minimum charge logic needed.

ParkingSpot

The methods here are trivial—just getters:

PARKINGSPOT METHODS

```
getSpotType()
    return spotType

getId()
    return id
```

ParkingSpot is a pure data holder. No state management, no occupancy tracking.

Ticket

Ticket is all getters:

TICKET METHODS

```
getId()
    return id

getSpotId()
    return spotId

getVehicleType()
    return vehicleType

getEntryTime()
    return entryTime
```

Pure data holder. No behavior.

Complete Code Implementation

While most companies only require pseudocode during interviews, some do ask for full implementations of at least a subset of the classes or methods. Below is a complete working implementation in common languages for reference.

ParkingLot.py | ParkingSpot.py | Ticket.py — Python

```python
import uuid
import time
from typing import List, Dict, Set

class ParkingLot:
    def __init__(self, spots: List, hourly_rate_cents: int):
        self._spots = spots
        self._active_tickets: Dict[str, Ticket] = {}
        self._occupied_spot_ids: Set[str] = set()
        self._hourly_rate_cents = hourly_rate_cents

    def enter(self, vehicle_type):
        spot = self._find_available_spot(vehicle_type)
```

```
            if spot is None:
                raise Exception(f"No available spots for vehicle type {vehicle_type}")

            self._occupied_spot_ids.add(spot.get_id())

            ticket_id = str(uuid.uuid4())
            entry_time = int(time.time() * 1000)
            ticket = Ticket(ticket_id, spot.get_id(), vehicle_type, entry_time)

            self._active_tickets[ticket_id] = ticket

            return ticket

        def exit(self, ticket_id: str) -> int:
```

Verification

Let's trace through a scenario to verify the state management works correctly. This catches bugs before your interviewer finds them.

ParkingLot has 3 spots: spot A (MOTORCYCLE), spot B (CAR), spot C (LARGE). occupiedSpotIds is empty. No active tickets. Hourly rate is $5 (500 cents).

Vehicle enters:

```
ENTER(CAR)

Initial: spots=[A, B, C], occupiedSpotIds={}, activeTickets={}

findAvailableSpot(CAR) → finds spot B (type matches, not in occupiedSpotIds)
occupiedSpotIds.add("B") → set now {"B"}
Generate ticket: id="T123", spotId="B", vehicleType=CAR, entryTime=1000000
activeTickets.put("T123", ticket) → map now {"T123" → ticket}

Return ticket T123
State: occupiedSpotIds={"B"}, activeTickets has T123
```

The spot is marked occupied in our Set and the ticket is stored.

Vehicle exits 2.5 hours later:

```
EXIT(
```

```
activeTickets.get("T123") → ticket found
exitTime = 1000000 + (2.5 hours in millis) = 10000000
computeFee(1000000, 10000000):
  - duration = 2.5 hours
  - round up → 3 hours
  - fee = 3 * 500 = 1500 cents

occupiedSpotIds.remove("B") → set now empty
activeTickets.remove("T123") → map now empty

Return 1500 cents
State: occupiedSpotIds={}, activeTickets={}
```

Fee correctly rounds up 2.5 to 3 hours. Spot is freed (removed from Set). Ticket is removed.

Try to exit again with same ticket:

EXIT(

```
activeTickets.get("T123") → null (already removed)
throw Error("Ticket not found or already used")
```

Double exit is prevented.

Try to enter when lot is full:

ENTER(CAR)

```
All CAR spots are in occupiedSpotIds
findAvailableSpot(CAR) → returns null
throw Error("No available spots for vehicle type CAR")
```

Entry is rejected without changing state.

This verifies the core workflows, state transitions, and error handling all work correctly.

Extensibility

If there's time left after implementation, interviewers often ask "what if" questions to see if your design can evolve cleanly. You typically won't implement these changes, just explain where they'd fit.

The depth of this section depends on your target level. Junior candidates often skip it. Mid-level candidates get one or two simple questions. Senior candidates might get several in a row.

> If you're a junior engineer, feel free to skip this section and stop reading here! Only carry on if you're curious about the more advanced concepts.

Below are the most common ones for parking lot systems, with more detail than you'd need in an actual interview.

1. "How would you extend this to a multi-floor parking garage?"

Our current design assumes a single floor with a flat list of spots. But what about a 10-floor garage at a shopping mall with thousands of spots? The interviewer wants to see if your design can scale without a complete rewrite.

> "The main change is introducing a ParkingFloor entity between ParkingLot and ParkingSpot. Each floor owns a collection of spots, and ParkingLot owns a collection of floors. The floor becomes part of the spot's identity, so spot IDs become something like '3-A15' for floor 3, section A, spot 15."

MULTI-FLOOR STRUCTURE

```
class ParkingLot:
    - floors: List<ParkingFloor>
    - activeTickets: Map<String, Ticket>
    - hourlyRateCents

class ParkingFloor:
    - floorNumber
    - spots: List<ParkingSpot>

    + getAvailableSpotCount(spotType) -> int
    + findAvailableSpot(spotType) -> ParkingSpot
```

The spot-finding logic now has options. The simplest approach is to iterate through floors in order and return the first available spot:

SIMPLE FLOOR ITERATION

```
findAvailableSpot(vehicleType)
    requiredType = mapVehicleTypeToSpotType(vehicleType)

    for floor in floors
        spot = floor.findAvailableSpot(requiredType)
        if spot != null
            return spot
    return null
```

But with 10 floors, you might want smarter allocation. You could even use a **Strategy** pattern to implement different allocation strategies based on occupancy or time of day.

1. Fill lower floors first — Keeps customers closer to the entrance/exit. This is what the simple iteration does.
2. Balance across floors — Spread vehicles evenly so no single floor gets congested. Track spot counts per floor and prefer floors with more availability.
3. Proximity to destination — In a mall garage, different floors might be closer to different stores. If someone says "I'm going to the food court," assign them to floor 4 which connects directly.

BALANCED ALLOCATION

```
findAvailableSpot(vehicleType)
    requiredType = mapVehicleTypeToSpotType(vehicleType)

    // Find floor with most available spots of this type
    bestFloor = null
    maxAvailable = 0

    for floor in floors
        available = floor.getAvailableSpotCount(requiredType)
        if available > maxAvailable
            maxAvailable = available
            bestFloor = floor

    if bestFloor == null
```

```
            return null
        return bestFloor.findAvailableSpot(requiredType)
```

The Ticket class doesn't change at all — it still stores the spotId, which now includes floor information implicitly (e.g., "3-A15"). The exit flow is identical: look up ticket, compute fee, free spot by ID, though you could also add a Floor state to the Ticket class to make it more explicit if you'd like.

2. "How would you add different pricing for different vehicle types?"

Right now we have one hourly rate for everyone. What if motorcycles are $3/hour, cars are $5/hour, and large vehicles are $8/hour?

> "There are two ways to do this. The simple way is to add a map from VehicleType to rate in ParkingLot. Store three rates instead of one. Then in `computeFee`, look up the rate based on the vehicle type from the ticket.
> The more sophisticated way, if we expect complex pricing rules, is to introduce a PricingStrategy interface with different implementations. But that's overkill unless we have very complex rules like surge pricing or discounts. For just three different rates, the map is simpler."

SIMPLE APPROACH

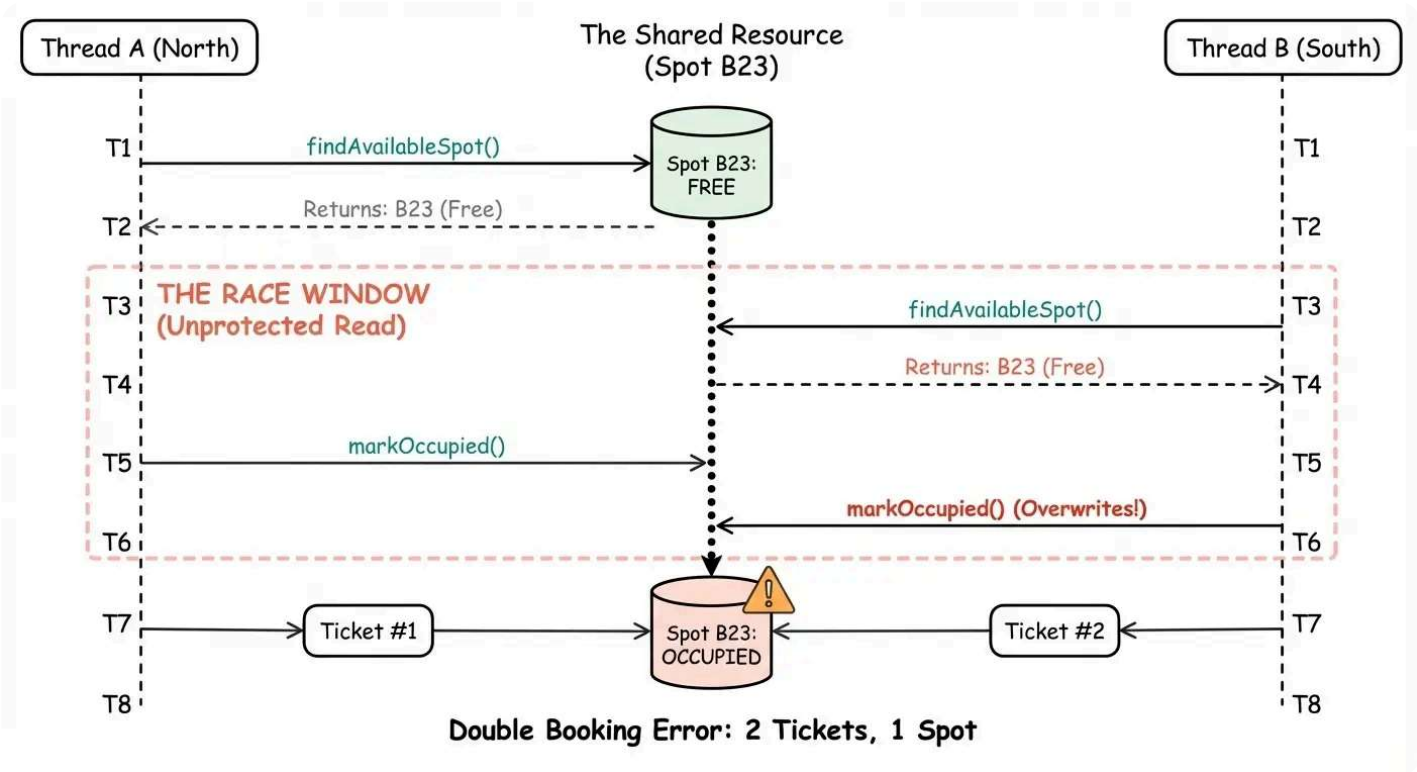
```
class ParkingLot:
  - spots: List<ParkingSpot>
  - activeTickets: Map<String, Ticket>
  - hourlyRates: Map<VehicleType, long>  // Change: map instead of single rate
```

```
computeFee(entryTime, exitTime, vehicleType)
    durationHours = calculateDuration(entryTime, exitTime)
    rate = hourlyRates[vehicleType]  // Look up rate by vehicle type
    return durationHours * rate
```

The signature of `computeFee` changes to take vehicle type (which we get from the ticket). The rest stays the same.

3. "How would you handle multiple entrances with concurrent access?"

If a large parking lot has multiple entrances, you can have two vehicles trying to enter at the exact same time. This creates a classic race condition where both threads could find the same spot available and try to assign it. Then both cars will have tickets for the same spot. Awkward!



Race condition when two vehicles enter simultaneously from different entrances

The window between checking if a spot is available and adding it to `occupiedSpotIds` is where the race happens. Let's look at how to fix this.

Good Solution: Coarse-Grained Lock (Synchronized Method)

Great Solution: Fine-Grained Read-Write Locking with Retry

> This discussion assumes synchronous, in-memory operations. If you're using async operations (database queries, API calls), single-threaded languages like Node.js can still have race conditions. The event loop can interleave requests at each `await` point, creating the same bug. In that case, you'd handle synchronization at the database level using transactions with proper isolation levels (SELECT FOR UPDATE in SQL) or application-level

> distributed locks (Redis, ZooKeeper). This is something discussed in System Design interviews, not Low Level Design typically.

So, as a candidate, you might respond with:

> "With multiple entrances, we have a race condition where two vehicles could be assigned the same spot. The simplest correct solution is to synchronize the entire `enter()` method, which serializes all entrance requests. This is sufficient for most parking lots. If we needed higher concurrency, we could use atomic check-and-add on the `occupiedSpotIds` Set with retry logic. For a parking lot with 3-5 entrances and typical traffic, method-level synchronization is the right choice—it's simple, correct, and performance isn't an issue."

What is Expected at Each Level?

So what am I looking for at each level?

Junior

At the junior level, I'm checking whether you can break down the problem and implement a working system. You should identify the need for something to represent spots, tickets, and an orchestrator. Your `enter` flow should find a spot, mark it occupied, and return a ticket. Your `exit` flow should calculate a fee and free the spot. Basic error handling matters: reject entry when full, reject invalid tickets. It's fine if you need hints on where to put pricing logic. What matters is building something that works.

Mid-level

For mid-level candidates, I expect cleaner separation of concerns without much guidance. `ParkingLot` orchestrates, `ParkingSpot` holds spot properties, `Ticket` is a simple data holder. You should recognize that `Vehicle` doesn't need to be a class (it's external, just a classification label). Handle the key edge cases: full lot, invalid tickets, double exits. You should justify design decisions when asked: why is `activeTickets` a map? Why is pricing in `ParkingLot` instead of on `Ticket`? If time allows, discuss at least one extension like multi-floor or per-type pricing.

Senior

Senior candidates should produce a design that demonstrates systems thinking. Class boundaries should be obvious without deliberation. You should proactively discuss trade-offs: for example, the occupied flag is controlled denormalization, tickets in a map enable O(1) lookup, and separate enums for `SpotType` and `VehicleType` allow for future flexibility or any other reasonable trade-off. I expect you to catch edge cases yourself. For extensibility, you should be able to discuss multiple approaches, explaining the simple solution first, and then when you'd reach for patterns like **Strategy**. Strong candidates finish early and can discuss how the design evolves for multi-floor garages or concurrent access.

Test Your Knowledge

Answer the question below to find your gaps.

Question 1 of 15

Why does the design use two separate enums (SpotType and VehicleType) even though they have the same values?

1 TypeScript requires separate enums for type safety

2 They are semantically distinct concepts

3 One is for the database and one for the API

4 To improve performance